GameYajok Document

Created by

Boonnapod Theeratassananon 6730276721

Phattaraphum Jamroensuk

6730407621

2110215 Programming Methodology

Semester 1 Year 2025

Chulalongkorn University

GameYajok

Introduction

This document is created to explain the structure of the Monopoly game program, describing the

responsibilities of each package and class, so it can be modified, extended, and used as a reference for submission.

Rule

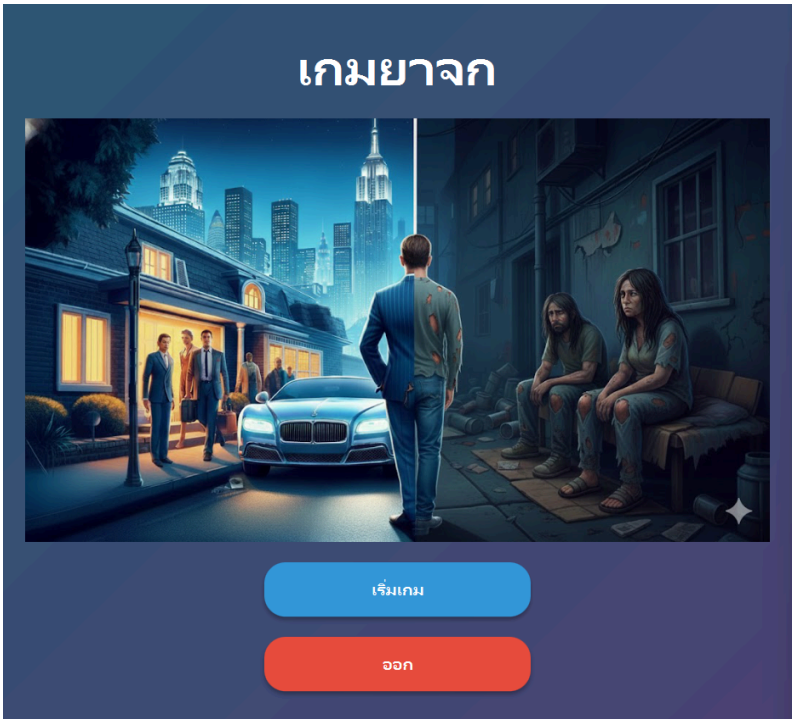
At the beginning of the game, each player will be able to choose their character. At the start of the game, both players receive an initial amount of $1000 each. The player must click to roll the dice, which has values from 1 to 6. After that, the player moves forward according to the number rolled on the dice.

1. If the player lands on an asset space that has no owner, the player may choose to buy it.
2. If the player lands on an asset space that they already own, they can build a landmark, which increases the rent.
3. If the player lands on an asset owned by the other player, they must pay rent and may choose whether or not to buy the asset.
4. If the player lands on a Mystery Card space, they will receive a random card and its ability will be applied immediately.
5. If the player lands on an Airport space, they can choose a destination to move to.
6. If the player lands on a Jail space, they must skip 2 turns. On the next turn, they may choose whether to pay $200 to get out of jail immediately.
7. If the player lands on a Lottery space, they will receive a random amount of money.
8. If the player lands on an Income Tax space, they must pay a random amount of tax.
9. When a player passes or lands on the starting point (the GO space), the player receives $100.

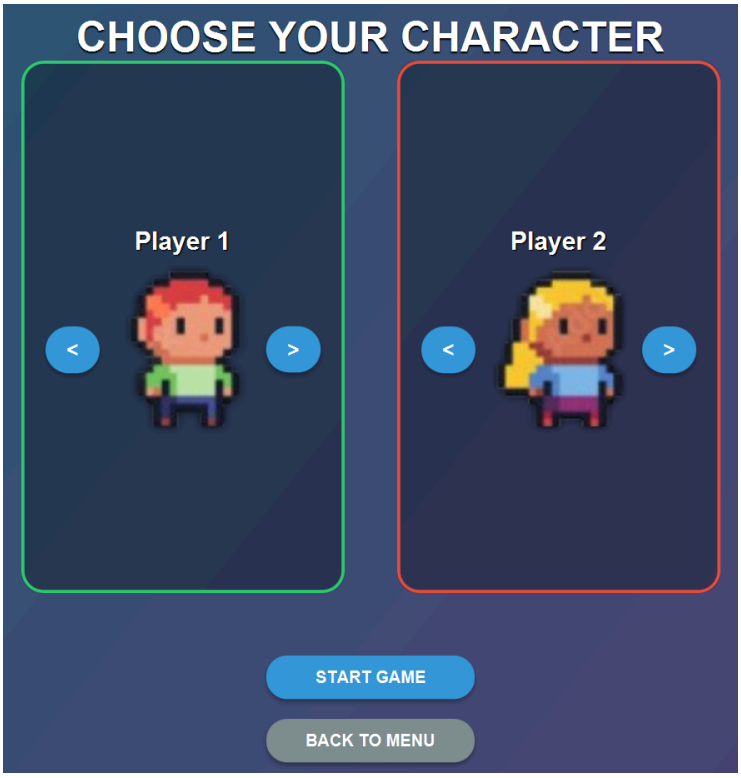
The person who goes bankrupt first is the winner.

Main Manu

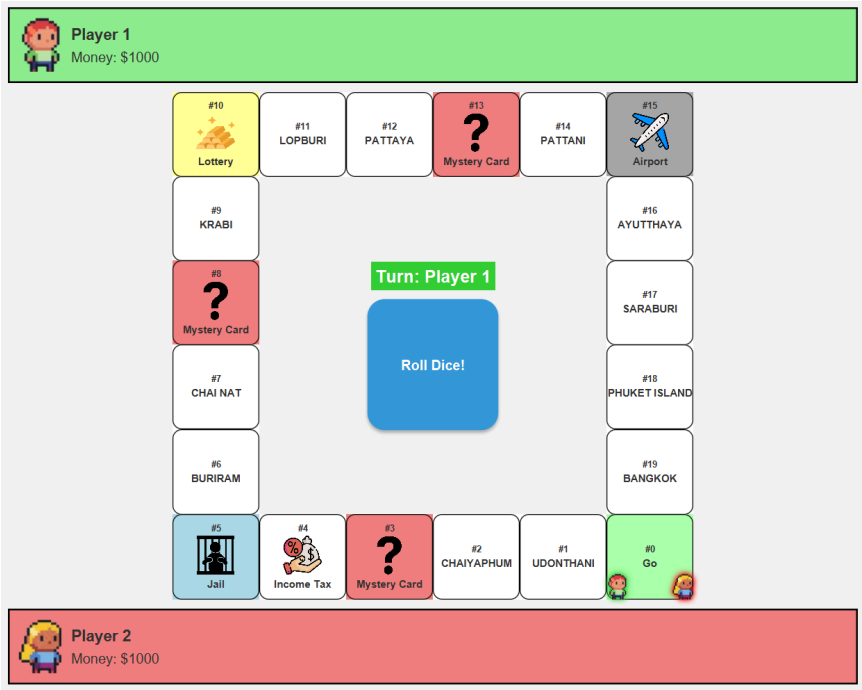
- Choose to start the game or exit.



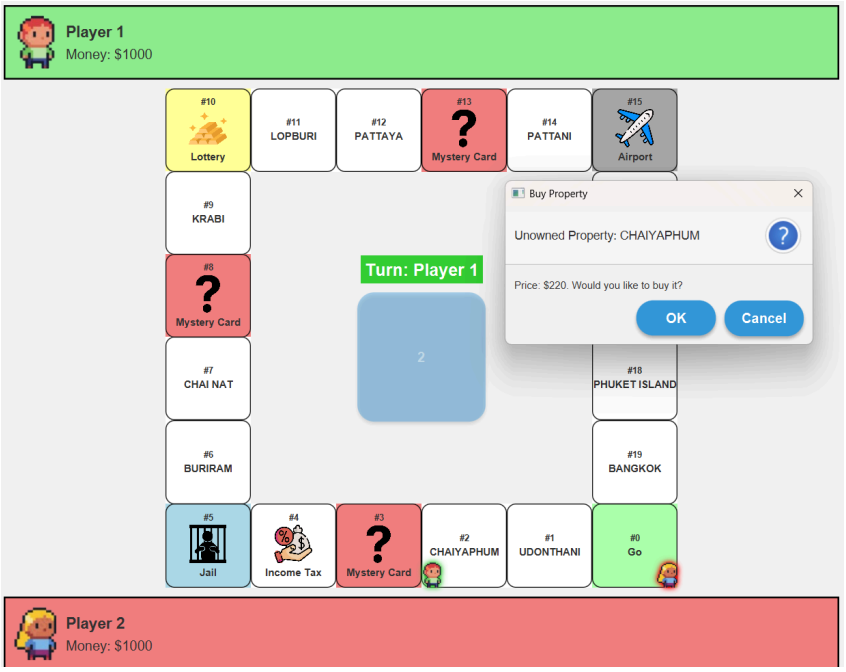
- Choose your character.



- After you click Start Game, this screen will appear.



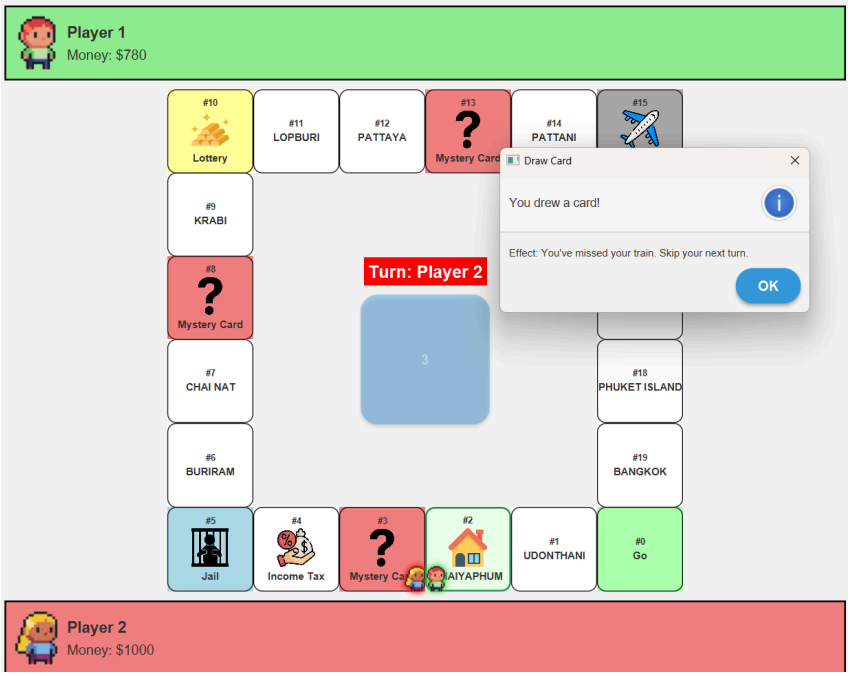
- When Roll Dice is clicked, the player moves according to the dice value, with Player 1 taking the first turn.
- If the player lands on an asset space that has no owner, the player may choose to buy it.



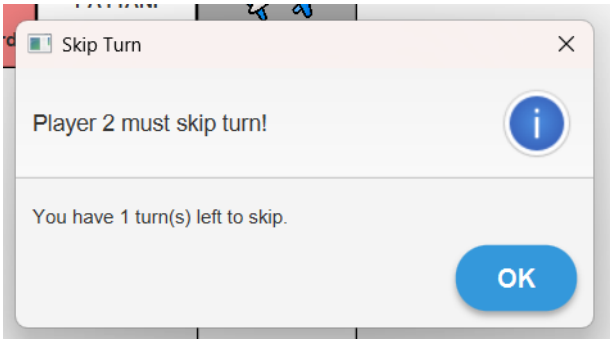
- After clicking OK, the square the player lands on will change to this.



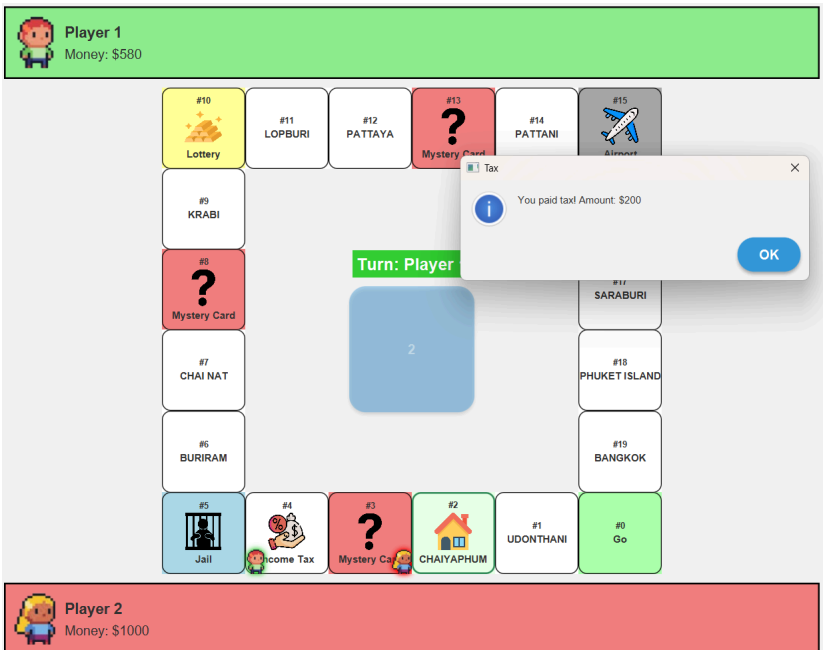
- If the player lands on a Mystery Card space, they will receive a random card and its ability will be applied immediately.



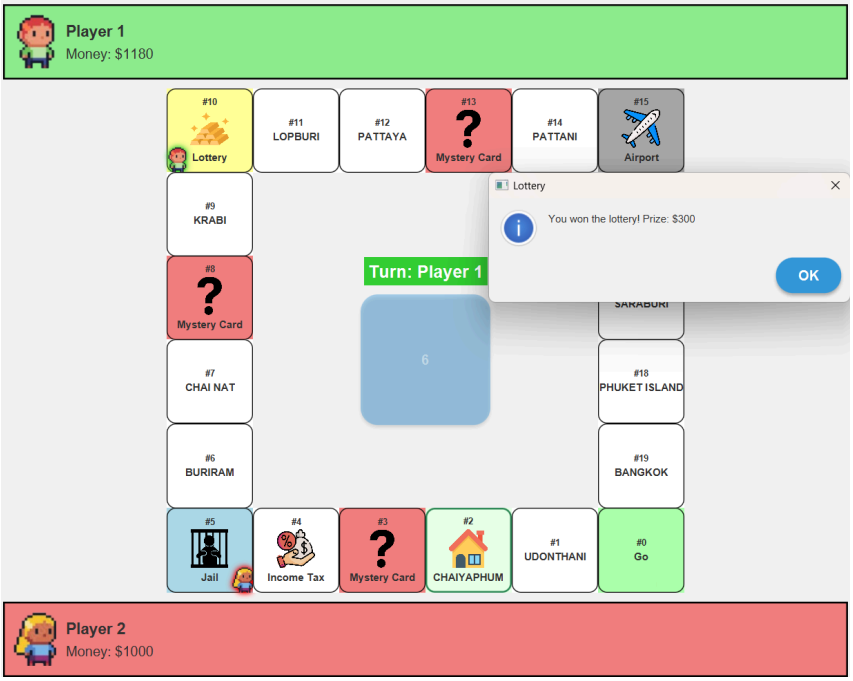
- It will show this in the next turn if you got a SkipTurnCard.



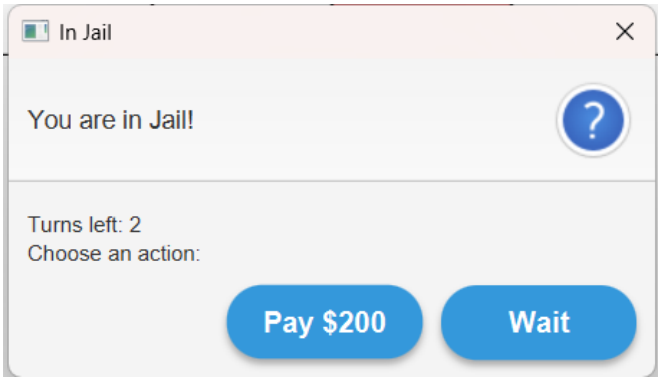
- If the player lands on an Income Tax space, they must pay a random amount of tax.



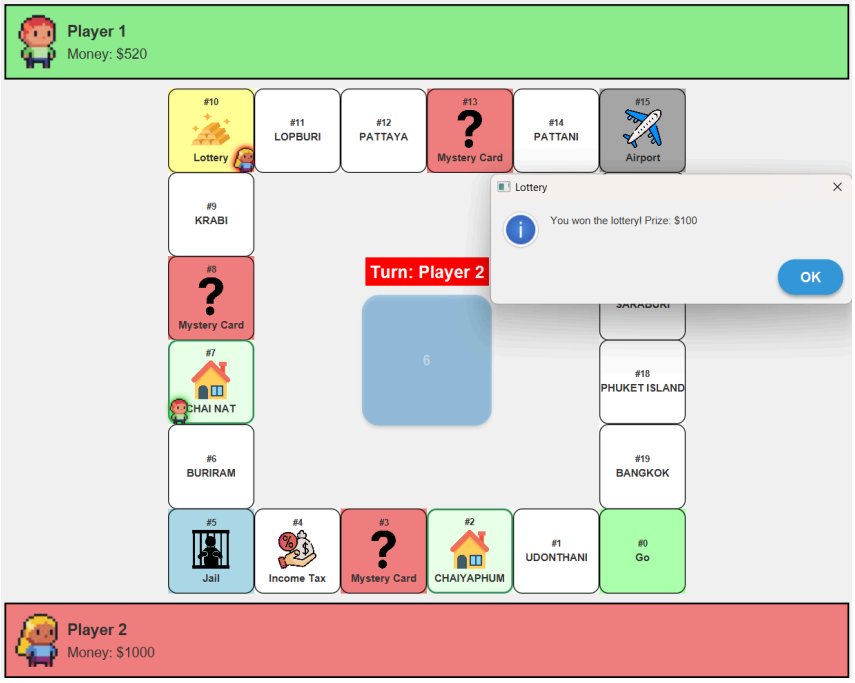
- If the player lands on a Jail space, they must skip 2 turns.



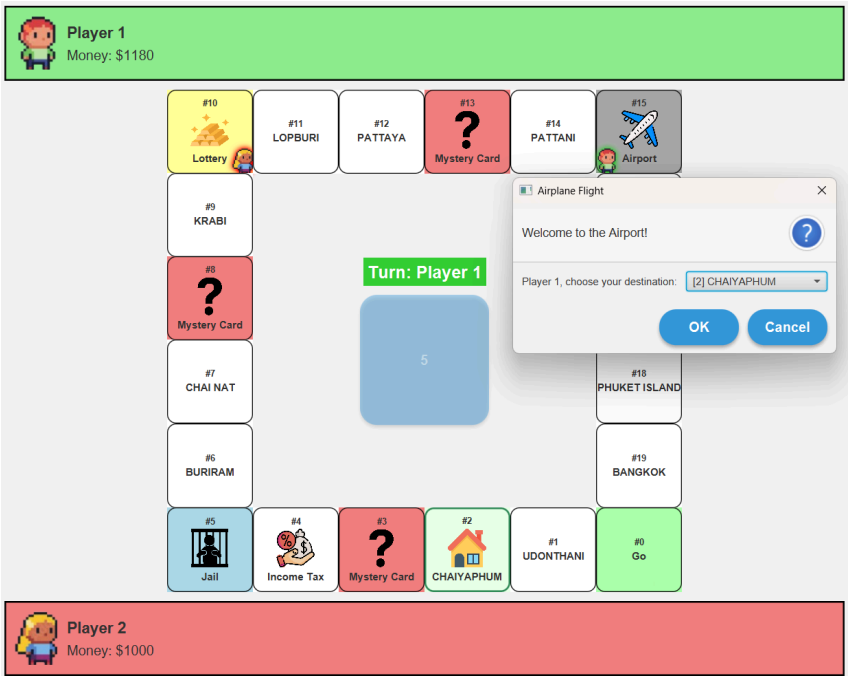
- This will appear on your next turn, and you can make your choice.
- If you choose to pay $200 to get out of jail immediately.



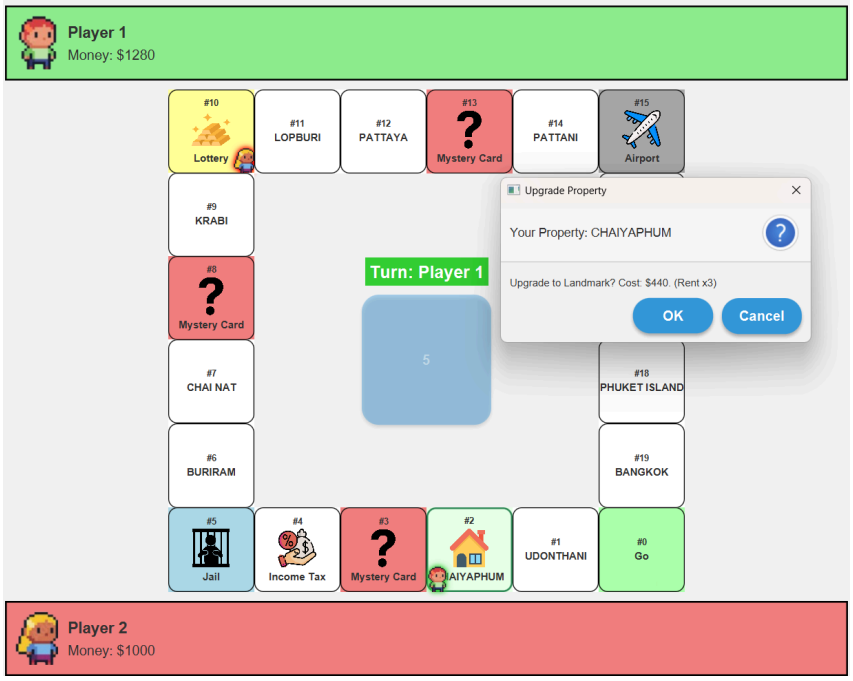
- If the player lands on a Lottery space, they will receive a random amount of money.



- If the player lands on an Airport space, they can choose a destination to move to.



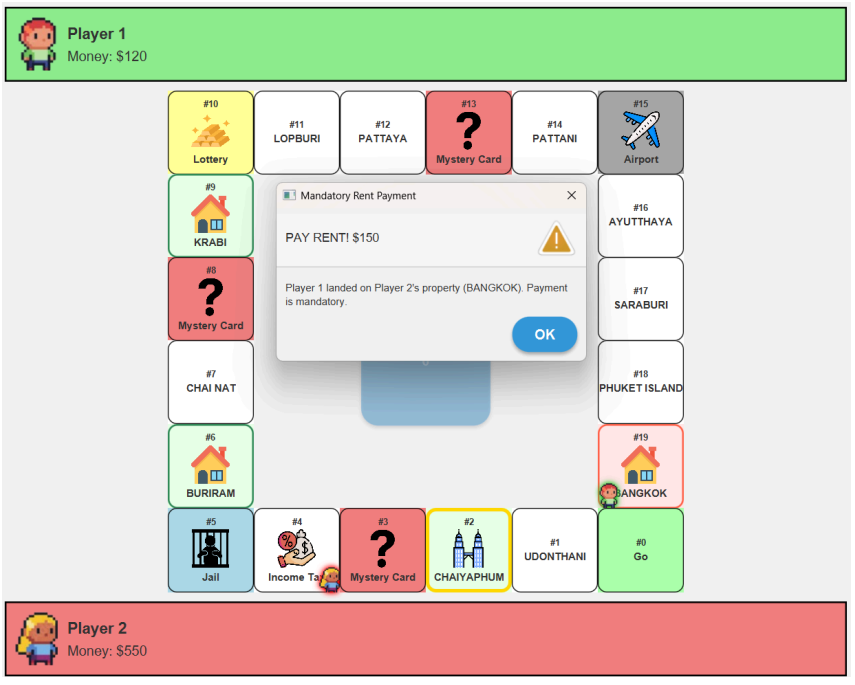
- If the player lands on an asset space that they already own, they can build a landmark, which increases the rent.



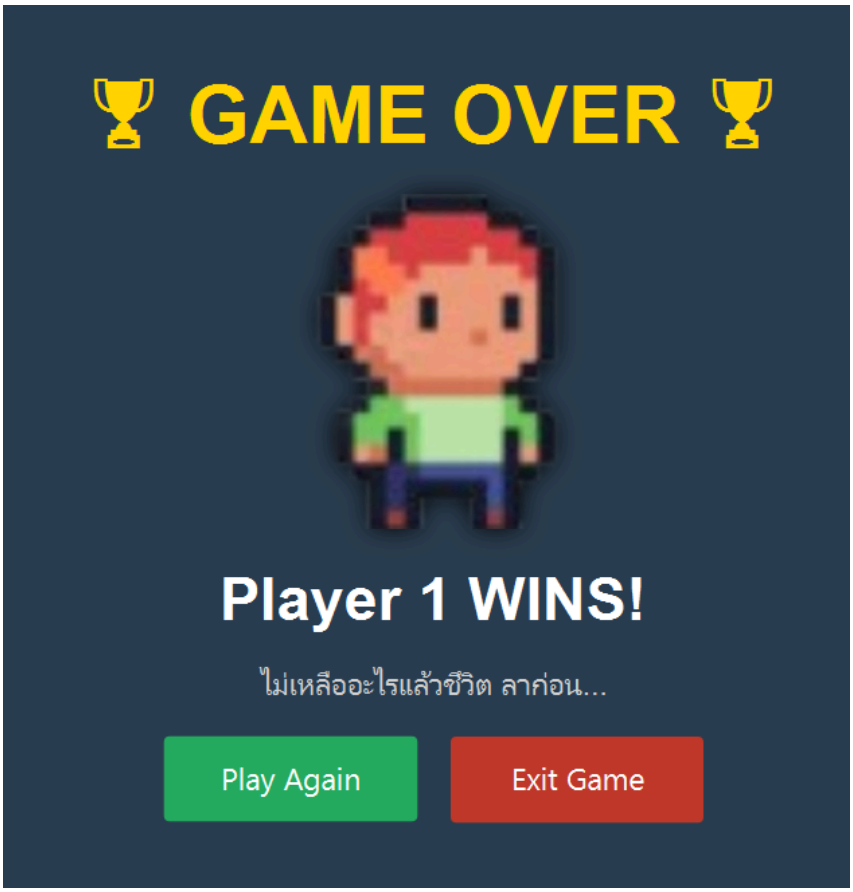
- After clicking OK, the square the player lands on will change to this.



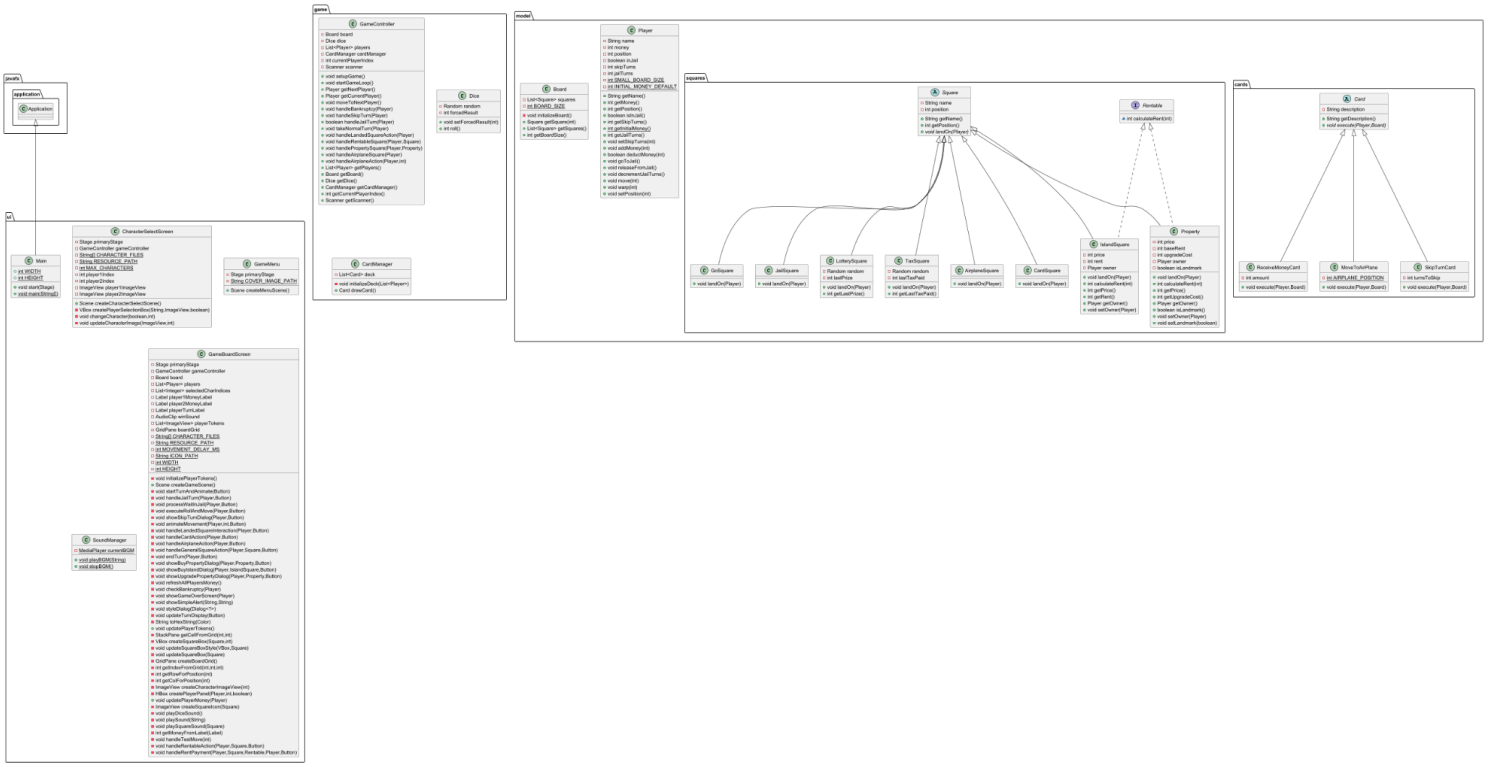
- If you land on a square that requires payment and you don't have enough money, you will go bankrupt and win.



- At the end of the game, this screen will be displayed.



Class Diagrams (All Class)

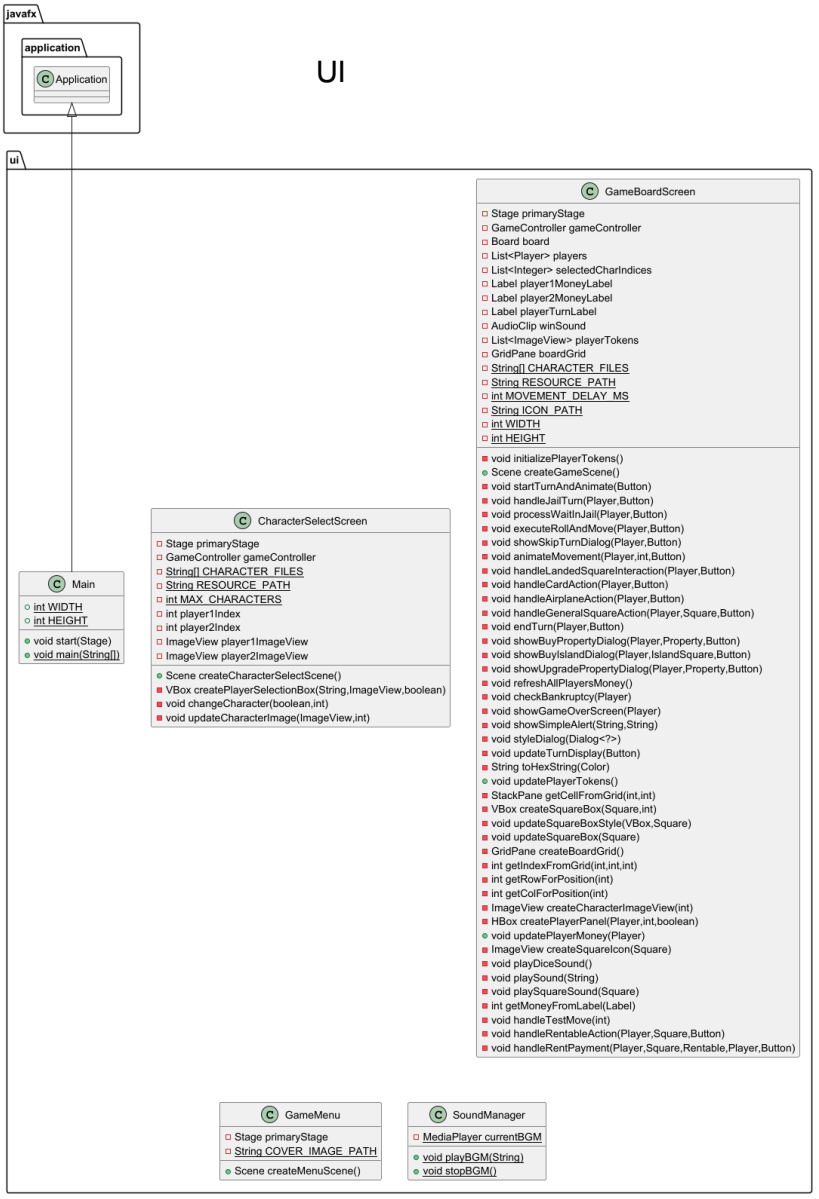


Class Diagrams

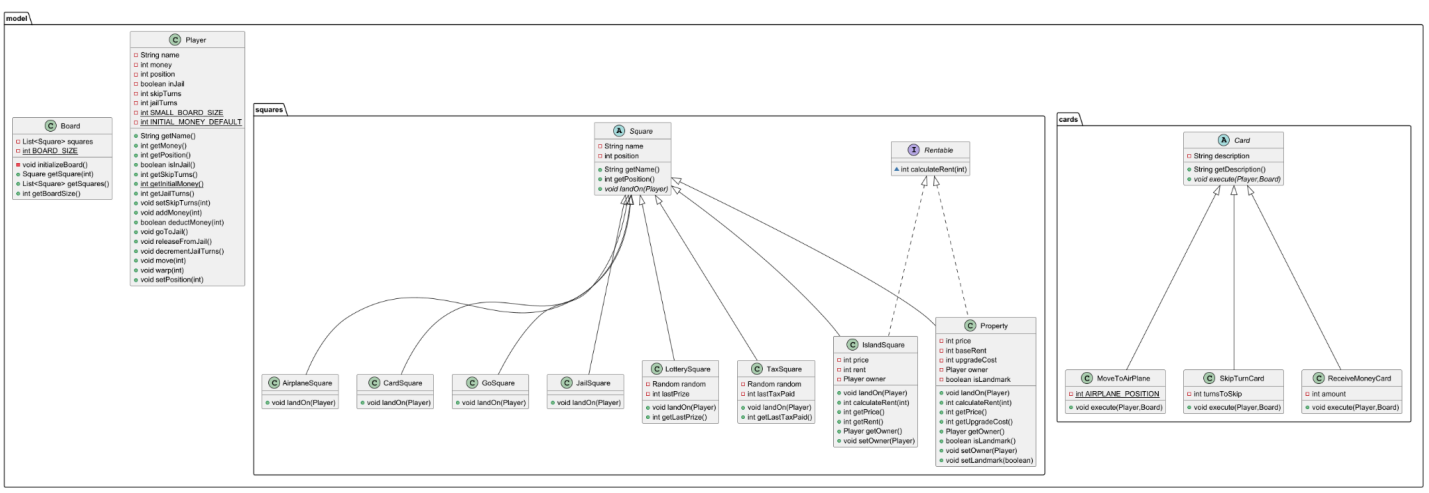
Game



UI



Model



1.Package : game

1.1 Class CardManager

Fields and Constructors

- final List<Card> deck	deck for cards
+ CardManager(List<Player> players)	1. Create a list to store the cards. 2. Add the cards into the deck. Shuffle the cards in the deck.

Methods

- void initializeDeck(List<Player> players)	Add cards into the deck. The cards have three abilities: 1. Move to the airport. 2. Skip one turn. 3. Receive a random amount of money equal to 10–20% of the starting money.
- Card drawCard()	Return the drawn card.

1.2 Class Dice

Fields and Constructors

- final Random random	This field is used to generate random values for the class.
+ Dice()	Used to define a random generator to support generating random numbers when rolling the dice.

Methods

+ int roll()	Return a random number from 1 to 6.

1.3 Class GameController

Fields and Constructors

- final Board board	Used to represent the game board and manage player positions within the game.
- final Dice dice	Used to generate random dice rolls to determine the number of spaces a player will move.
- final List<Player> players	Used to store information of all players participating in the game and to manage the turn order.
- final CardManager cardManager	Used to manage the card deck, shuffle the cards, and draw a card when a player lands on a special space.

- int currentPlayerIndex	Used to control the turn order of players in each round.
- final Scanner scanner	Used to receive input from the user via the keyboard, such as commands for gameplay or confirmation of various actions within the game.
+ GameController()	1. Create board, dice, and scanner objects. 2. Create a players list and add the initial 2 players ("Player 1" and "Player 2") to the list. 3. Create cardManager using the list of players. 4. Set currentPlayerIndex to 0.

Methods

+ void setupGame()	Display the game start message.
+ void startGameLoop()	Start the game, looping continuously until only one player remains, then exit the loop to display the game over message.
+ Player getNextPlayer()	1. Check bounds: if currentPlayerIndex exceeds the number of players, reset it to 0 2. Return player: if the players list is not empty, return the player at the currentPlayerIndex immediately
+ Player getCurrentPlayer()	Return the current player
+ void moveToNextPlayer()	Switch the turn to the next player
+ void handleBankruptcy(Player player)	1. Display bankruptcy message and congratulate the winner 2. Remove the bankrupt player's houses and the player from the game
+ void handleSkipTurn(Player player)	Skip the player's turn and display a message
+ boolean handleJailTurn(Player player)	1. Display a message asking the player if they want to pay $50 to get out of jail 2. Receive input from the user (y/n) 3. If the answer is 'y' and the player has enough money: deduct $50, release the player from jail (releaseFromJail), and return true (allowing the player to take their turn normally) 4. If the player does not have enough money or answers 'n': display a message that the player remains in jail and return false
+ void takeNormailTurn(Player player)	Roll the dice and move the player according to the number rolled
+ void handleLandedSquareAction(Player player)	1. CardSquare: draw a card and execute its effect. If the card effect moves the player, this method is called recursively to handle the new square, which may become an

	AirplaneSquare 2. Property: call the handlePropertySquare method 3. AirplaneSquare: call the handleAirplaneSquare method 4. Other squares: call the landOn method of that square
+ void handlePropertySquare(Player player, Property property)	1. If owned by another player: calculate and deduct rent from the player to pay the owner (if the player cannot pay, they go bankrupt). After paying rent, if the property is not yet a Landmark, offer the option to buy it at 1.5 times the original price (Buyout). 2. If owned by the player: if the property is not yet a Landmark, offer the option to upgrade it to a Landmark (rent increases 3 times). 3. If unowned: offer the property for sale to the player if they have enough money.
+ void handleAirplaneSquare(Player player)	1. Displays the list of all available destination squares (0-19) on the console. - 2. Receives and validates user input for the target position (ensuring it is an integer between 0-19). 3. Calls handleAirplaneAction to move the player to the chosen destination. 4. Calls handleLandedSquareAction to immediately trigger the logic of the new square the player landed on.
+ void handleAirplaneAction(Player player, int targetPosition)	1. Calculate the number of steps to move forward from the current position to the target (if the target is behind, calculate by looping around the board). 2. Call player.move to move, which automatically gives the player money if they pass the starting point.
Remaining getter/setter	

2.Package : model

2.1 package model.cards

2.1.1 Abstract Class Card

Fields and Constructors

- final String description	Set the text for the card

+ Card(String description)	Set the message or text on the card

Method

+ void execute(Player player, Board board)	The child class must implement this to define the specific behavior of each card

2.1.2 Class MoveToAirPlane extends Card

Fields and Constructors

- static final int AIRPLANE_POSITION	Constant indicating the index position of the airplane square on the board (set to 15)
+ MoveToAirPlane(String description)	Set the description through the parent class (Card) constructor

Method

+ void execute(Player player, Board board)	- Displays the card's action on the console. - Calculates the number of steps required to move forward from the current position to the AIRPLANE_POSITION (calculating a wrap-around if the target is behind the current position). - Calls the currentPlayer.move(steps) method to transport the player (this ensures the logic for collecting money when passing 'Go' is triggered correctly). - Displays the player's new position.

2.1.3 Class ReceiveMoneyCard extends Card

Fields and Constructors

- final int amount	The amount of money the player will receive when this card is executed.
+ ReceiveMoneyCard(String description, int amount)	Constructor method. Initializes the card with the specified description and sets the amount of money to be received.

Method

+ void execute(Player player, Board board)	1. Displays the card's description on the console. 2. Adds the specified amount to the player's current money balance. 3. Prints a confirmation message indicating the player collected the money.

2.1.4 Class SkipTurnCard extends Card

Fields and Constructors

- final int turnsToSkip	The number of turns the player is required to skip when this card is executed.
+ SkipTurnCard(String description, int turnsToSkip)	Constructor method. Initializes the card with the specified description and sets the number of turns the player is required to skip.

Method

+ void execute(Player player, Board board)	1. Displays the card's description on the console. 2. Increases the player's skip turn count by adding the value of turnsToSkip. 3. Prints a message indicating the total number of turns the player must now skip.

2.2 package model.squares

2.2.1 Class AirplaneSquare extends Square

Constructors

+ AirplaneSquare(String name, int position)	Constructor method. Initializes the AirplaneSquare object with the specified name and position by calling the superclass constructor.

Method

+ void landOn(Player player)	1. Prints a message to the console indicating that the player has landed on the square. 2. Displays a prompt message instructing the player to choose a destination (Note: The actual logic for selecting the destination is handled externally in the GameController).

2.2.2 Class CardSquare extends Square

Constructors

+ CardSquare(String name, int position)	Constructor method. Initializes the CardSquare object with the specified name and position by calling the superclass constructor.

Method

+ void landOn(Player player)	Prints a message to the console indicating that the player has landed on the square and should prepare to draw a card (Note: The actual card drawing logic is handled externally in the GameController).

2.2.3 Class GoSquare extends Square

Constructors

+ GoSquare(int position)	Constructor method. Initializes the GoSquare object with the name "Go" and the specified position by calling the superclass constructor.

Method

+ void landOn(Player player)	Prints a message to the console indicating that the player has landed on "Go" (Note: The logic for collecting money when passing/landing on Go is handled automatically within the Player.move() method).

2.2.4 Class JailSquare extends Square

Constructors

+ JailSquare(String name, int position)	Constructor method. Initializes the JailSquare object with the specified name and position by calling the superclass constructor.

Method

+ void landOn(Player player)	Prints a message to the console indicating that the player is in Jail.

2.2.5 Class LotterySquare extends Square

Fields and Constructors

- final Random random	A Random object instance used to generate random numbers for determining the lottery prize amount.
- int lastPrize	Stores the value of the most recently won prize amount for display or retrieval purposes (initialized to 0).
+ LotterySquare(String name, int position)	Constructor method. Initializes the LotterySquare object by calling the superclass constructor with the specified name and position, and initializes the random object.

Method

+ void landOn(Player player)	1. Generates a random lucky number between 1 and 5, then multiplies it by 100 to determine the lastPrize. 2. Prints messages to the console indicating the player landed on the square and the amount won. 3. Adds the calculated prize amount to the player's money

	balance using player.addMoney().
+ int getLastPrize()	Returns the value of lastPrize, representing the most recently won prize amount.

2.2.6 Class Property extends Square implements Rentable

Fields and Constructors

- final int price	The purchase price of the property.
- final int baseRent	The base amount of rent to be paid when landing on this property.
- final int upgradeCost	The cost required to upgrade the property to a Landmark.
- Player owner	The current owner of the property (initially set to null).
- boolean isLandmark	Indicates whether the property has been upgraded to a Landmark (default is false).
+ Property(String name, int position, int price, int baseRent)	Calls the superclass constructor to set the name and position.

Method

+ int calculateRent(int diceRoll)	Implementation of the Rentable interface. Calculates the rent amount to be paid. Returns the baseRent, but if the property is a Landmark (isLandmark is true), the rent is multiplied by 3.
Remaining getter/setters for the fields	

2.2.7 Interface Rentable

Constructors

int calculateRent(int diceRoll)	Calculates the rent fee that the player is required to pay. Returns the calculated rent amount.

2.2.8 Abstract Class Square

Constructors

- final String name	The name of the square, used for display purposes.
- final int position	The position index of the square on the board (0-19).
+ Square(String name, int position)	Constructor method. Initializes the square with the specified name and position.
+ String getName()	Getter method. Returns the name of the square.

+ int getPosition()	Getter method. Returns the position index of the square.

Method

+ void landOn(Player player)	Abstract method. Must be implemented by subclasses to define the specific behavior when a player lands on the square (Polymorphism).

2.2.9 Class TaxSquare extends Square

Constructors

- final Random random	A Random object instance used to generate random numbers for determining the variable tax amount.
- int lastTaxPaid	Stores the value of the most recently paid tax amount for display or retrieval purposes (initialized to 0).
+ TaxSquare(String name, int position)	Constructor method. Initializes the TaxSquare object by calling the superclass constructor with the specified name and position, and initializes the random object.

Method

+ void landOn(Player player)	1. Calculates a random tax amount ranging from 100 to 300 (in increments of 50). 2. Stores the calculated amount in lastTaxPaid. 3. Prints messages to the console indicating the tax payable. 4. Deducts the tax amount from the player's money using player.deductMoney().
+ int getLastTaxPaid()	Returns the value of lastTaxPaid, representing the most recently calculated tax amount.

2.3 Class Board

Constructors

- final List<Square> squares	A list containing all the Square objects that constitute the game board.
- static final int BOARD_SIZE	A constant integer representing the total number of squares on the board (assigned as 20).
+ Board()	Constructor method. Initializes the squares list with the defined BOARD_SIZE capacity.

Method

- void initializeBoard()	1. Initializes the squares list with 20 null elements. 2. Assigns special squares to specific positions. 3. Configures the Jail square at position 5 using an anonymous class to immediately arrest players upon landing. 4. Initializes Property squares with specific prices and rents at their respective indices. 5. Fills any remaining empty slots with generic properties to ensure no null values exist.
+ Square getSquare(int index)	Returns the Square object at the specified index, using modulo arithmetic (% BOARD_SIZE) to ensure the index wraps around the board correctly.
Remaining getter/setters for the fields	

2.4 Class Player

Constructors

- final String name	The name of the player.
- int money	The current amount of money the player possesses.
- int position	The current position of the player on the board (ranging from 0 to 19).
- boolean inJail	Indicates whether the player is currently in jail.
- int skipTurns	The number of turns the player must skip.
- int jailTurns	The remaining number of turns the player must stay in jail (initialized to 0).
- static final int SMALL_BOARD_SIZE	A constant representing the total number of squares on the board (20).
- static final int INITIAL_MONEY_DEFAULT	A constant representing the starting money for a player (2000).
+ Player(String name)	Constructor method. Initializes a new player with the specified name, sets the initial money to INITIAL_MONEY_DEFAULT, places the player at position 0, and resets all status counters.

Method

+ void addMoney(int amount)	Increases the player's money by the specified amount and prints a message showing the received amount and the new balance.
+ boolean deductMoney(int amount)	Attempts to deduct money from the player:

	1. If the player has sufficient funds, deducts the amount and returns true. 2. If funds are insufficient, deducts the amount (resulting in a negative balance/bankruptcy) and returns false.
+ void goToJail()	Sends the player to jail by performing the following: 1. Sets the inJail status to true. 2. Moves the player to position 5 (Jail square). 3. Sets the jailTurns counter to 2.
+ void releaseFromJail()	Releases the player from jail by setting the inJail status to false and resetting the jailTurns counter to 0.
+ void decrementJailTurns()	Decreases the jailTurns counter by 1 (if greater than 0), used when the player chooses to wait out their sentence.
+ void move(int steps)	Moves the player forward by the specified number of steps using modulo arithmetic. Checks if the player passes the 'Go' square (index 0); if so, adds $200 to their balance.
+ void warp(int position)	Directly sets the player's current position to the specified position. This method is used for teleportation (e.g., Airplane travel) and does not trigger standard movement logic, such as collecting money for passing 'Go'.
Remaining getter/setters for the fields	

3.Package : UI

3.1 Class CharacterSelectScreen

Fields and Constructors

- final Stage primaryStage	The main window (stage) of the application where the scene will be displayed.
- final GameController gameController	A reference to the main GameController instance, used to pass game data to the next screen.
- static final String[] CHARACTER_FILES	An array containing the file names of the character images available for selection.
- static final String RESOURCE_PATH	The relative resource path to the directory containing character images.
- static final int MAX_CHARACTERS	The total number of available characters, derived from the length of the CHARACTER_FILES array.
- int player1Index	The index of the character currently selected by Player 1 (initialized to 0).

- int player2Index	The index of the character currently selected by Player 2 (initialized to 0).
- final ImageView player1ImageView	The JavaFX component used to display the selected character image for Player 1.
- final ImageView player2ImageView	The JavaFX component used to display the selected character image for Player 2.
+ CharacterSelectScreen(Stage primaryStage, GameController gameController)	Constructor method. Initializes the CharacterSelectScreen with the provided primaryStage and gameController instances.

Method

+ Scene createCharacterSelectScene()	1. Starts playing background music ("select.mp3"). 2. Initializes the UI layout, including the header label, player selection boxes, and initial character images. 3. Configures the "START GAME" button to collect selected character data, stop the background music, and transition to the GameBoardScreen. 4. Configures the "BACK TO MENU" button to stop the background music and return to the GameMenu scene. 5. Applies the CSS stylesheet and returns the constructed Scene object.
- VBox createPlayerSelectionBox(String titleText, ImageView imageView, boolean isPlayer1)	1. Creates and styles the title label with specific font settings and shadow effects. 2. Configures the ImageView with fixed dimensions (150x150) and a drop shadow. 3. Creates "Previous" and "Next" buttons that trigger the changeCharacter method when clicked. 4. Arranges the buttons and image into a horizontal layout (HBox). 5. Styles the main container with a semi-transparent background and a specific border color (Green for Player 1, Red for Player 2).
- void changeCharacter(boolean isPlayer1, int direction)	1. Calculates the new character index based on the direction (+1 or -1), ensuring the index wraps around correctly (circular selection). 2. Updates the corresponding index field (player1Index or player2Index). 3. Calls updateCharacterImage to visually refresh the player's character display.
- void updateCharacterImage(ImageView imageView, int index)	1. Retrieves the filename from CHARACTER_FILES using the provided index. 2. Loads the image from the resource path and sets it to the target ImageView.

3.2 Class GameBoardScreen

Fields and Constructors

- final Stage primaryStage	The main window (stage) of the application where the game scene will be displayed.
- final GameController gameController	A reference to the main GameController instance, used to handle game logic and state updates.
- final Board board	The game board model retrieved from the controller, containing data about all squares.
- final List<Player> players	A list containing all the Player objects participating in the current game session.
- final List<Integer> selectedCharIndices	A list of integers representing the indices of the character images selected by each player.
- Label player1MoneyLabel	UI label used to display the current money balance of Player 1.
- Label player2MoneyLabel	UI label used to display the current money balance of Player 2.
- Label playerTurnLabel	UI label used to indicate which player's turn it is currently.
- AudioClip winSound	The audio clip object used to play and control the winning sound effect.
- final List<ImageView> playerTokens	A list of ImageView objects representing the visual tokens (characters) for each player on the board.
- final GridPane boardGrid	The JavaFX grid layout responsible for visually arranging the game board squares.
- static final String[] CHARACTER_FILES	An array containing the file names of the available character images.
- static final String RESOURCE_PATH	The relative resource path to the directory containing character images.
- static final int MOVEMENT_DELAY_MS	A constant defining the delay in milliseconds (300ms) for the player movement animation.
- static final String ICON_PATH	The relative resource path to the directory containing icon images.
- static final int WIDTH	The constant width of the game screen (1000 pixels).

- static final int HEIGHT	The constant height of the game screen (800 pixels).
+ GameBoardScreen(Stage primaryStage, GameController gameController, List<Player> players, List<Integer> selectedCharIndices)	1. Assigns the passed primaryStage, gameController, players, and selectedCharIndices to the class fields. 2. Retrieves the board from the game controller. 3. Initializes the playerTokens list. 4. Calls createBoardGrid() to generate the visual board layout. 5. Calls initializePlayerTokens() to setup the visual representation of players.

Methods

- void initializePlayerTokens()	Sets up the visual tokens for all players by performing the following actions: 1. Defines an array of shadow colors (LimeGreen and Red) to visually distinguish the players. 2. Iterates through the list of players to create tokens. 3. Creates an ImageView for each player's selected character, setting the dimensions to 30x30 pixels. 4. Applies a specific DropShadow effect to the token corresponding to the player's index. 5. Adds the fully configured token to the playerTokens list for tracking.
+ Scene createGameScene()	Constructs and returns the main game scene by performing the following actions: 1. Starts playing the gameplay background music ("board.mp3"). 2. Initializes the root layout (BorderPane) and applies the background style. 3. Creates the "Roll Dice!" button and configures its event handler to disable the button, trigger the turn animation logic, and play the dice sound when clicked. 4. Creates a label to display the name of the player whose turn it currently is. 5. Assembles the central layout by stacking the game board grid and the control elements (Button and Label) together. 6. Generates and positions the player information panels (Top for Player 1, Bottom for Player 2). 7. Calls updatePlayerTokens() and updateTurnDisplay() to render the initial game state. 8. Loads the CSS stylesheet and returns

	the constructed Scene object.
- void startTurnAndAnimate(Button rollButton)	Initiates the logic for the current player's turn by checking their status before allowing movement: 1. Checks if the player is currently in jail. If true, delegates to handleJailTurn. 2. Checks if the player is required to skip a turn. If true, calls showSkipTurnDialog. 3. If the player is free to move, calls executeRollAndMove to begin the dice rolling sequence.
- void handleJailTurn(Player player, Button rollButton)	Manages the interaction when a player starts their turn in jail: 1. Displays a dialog offering the player a choice to either "Pay $200" or "Wait". 2. If "Pay" is selected: Attempts to deduct the bail amount. If successful, releases the player and immediately allows them to roll (executeRollAndMove). If funds are insufficient, defaults to the waiting logic. 3. If "Wait" is selected: Calls processWaitInJail to reduce the remaining sentence.
- void processWaitInJail(Player player, Button rollButton)	Handles the logic for a player who chooses to wait out their jail sentence: 1. Decrements the player's remaining jail turns count. 2. Checks if the sentence is complete (turns <= 0). If so, releases the player for the next turn. 3. Displays a message indicating the remaining turns and calls endTurn to finish the player's action.
- void executeRollAndMove(Player activePlayer, Button rollButton)	Executes the visual and logical dice rolling process: 1. Plays the dice rolling sound effect. 2. Runs a Timeline animation to simulate the dice rolling by rapidly changing numbers on the button. 3. Upon animation completion, generates the actual dice result from the GameController. 4. Updates the button with the final number and triggers the movement animation (animateMovement) after a short delay.
- void showSkipTurnDialog(Player player, Button rollButton)	Manages the interaction when a player must skip their turn: 1. Creates and styles an information dialog informing the player of the skip.

	2. Decrements the player's skipTurns counter by 1. 3. Displays the dialog and waits for user acknowledgement. 4. Advances the game to the next player via the controller and updates the UI.
- void animateMovement(Player player, int steps, Button rollButton)	Handles the visual movement of the player token: 1. Creates a Timeline to move the player step-by-step with a delay. 2. On each step: Updates the logical position (warp), checks if the player passed 'Go' (adding money if true), and updates the token's visual position. 3. On completion: Triggers handleLandedSquareInteraction, plays the sound effect for the specific square, and re-enables the roll button.
- void handleLandedSquareInteraction(Player player, Button rollButton)	The central hub for processing the result of landing on a square: 1. Retrieves the target square and executes its model-level logic (landOn). 2. Refreshes the money display for all players. 3. Identifies the type of square (Property, Card, Airplane, etc.) and delegates the UI interaction to the specific handler method (e.g., handlePropertyAction).
- void handlePropertyAction(Player player, Property property, Button rollButton)	Manages complex interactions when landing on a Property square: 1. If owned by another: Calculates rent, shows a payment dialog, and transfers money. Then, checks if a "Buyout" is possible (money > price * 1.5 & not a landmark); if so, offers the buyout option. 2. If owned by self: Checks if the property is not yet a Landmark; if so, opens the Upgrade dialog. 3. If unowned: Opens the Buy Property dialog. 4. Calls endTurn if no further action is taken.
- void handleCardAction(Player player, Button rollButton)	Manages the interaction when a player lands on a Card square: 1. Draws a card using the CardManager and displays its description in a dialog. 2. Executes the card's effect and updates the UI (money and tokens). 3. Checks if the card moved the player to a new position. If so, recursively calls

	handleLandedSquareInteraction to process the new square immediately; otherwise, ends the turn.
- void handleAirplaneAction(Player player, Button rollButton)	Manages the interaction for the Airplane/Airport square: 1. Generates a list of all board squares and displays them in a ChoiceDialog. 2. If a destination is selected: Moves the player via the controller, updates the UI, and recursively calls handleLandedSquareInteraction for the new location. 3. If cancelled: Displays a message and ends the turn.
- void handleGeneralSquareAction(Player player, Square landedSquare, Button rollButton)	Handles logic for standard squares that do not require complex inputs: \1. Identifies the square type (Tax, Lottery, or Jail). 2. Displays an informational alert specific to the square (e.g., tax amount paid, lottery prize won, or jail status). 3. Calls endTurn to conclude the player's action.
- void endTurn(Player player, Button rollButton)	Finalizes the current player's turn: 1. Checks if the player has gone bankrupt (checkBankruptcy). 2. Signals the GameController to advance to the next player. 3. Updates the UI display to show the next player's turn.
- void showBuyPropertyDialog(Player player, Property property, Button rollButton)	Handles the UI and logic for purchasing a property: 1. Displays a confirmation dialog showing the property name and price. 2. If the user confirms: Attempts to deduct the price from the player's funds. 3. If successful: Sets the player as the owner and updates the square's visual appearance. 4. If funds are insufficient: Displays an error alert. 5. Refreshes the money display and ends the turn.
- void showUpgradePropertyDialog(Player player, Property property, Button rollButton)	Handles the UI and logic for upgrading a property to a Landmark: 1. Displays a confirmation dialog showing the upgrade cost and the benefit (Rent x3). 2. If the user confirms: Attempts to deduct the upgrade cost. 3. If successful: Sets the property's status

	to Landmark and updates the visual appearance. 4. Refreshes the money display and ends the turn.
- void refreshAllPlayersMoney()	Helper method. Iterates through the list of players and calls updatePlayerMoney for each one to ensure the UI labels reflect the current balance of every player.
- void checkBankruptcy(Player player)	Checks the financial status of the player. If the player's money drops to zero or below, it triggers the showGameOverScreen method with the current player (or the opponent) as the winner context.
- void showGameOverScreen(Player winner)	transitions the application to the Game Over screen: 1. Plays the victory background music ("win.mp3"). 2. Constructs a new UI layout containing a "GAME OVER" header, the winner's name, and their character image. 3. Configures the "Play Again" button to return users to the CharacterSelectScreen. 4. Configures the "Exit Game" button to close the application. 5. Sets the primary stage to display this new scene.
- void showSimpleAlert(String title, String content)	A helper method that displays a simple information dialog: 1. Creates an Alert of type INFORMATION. 2. Sets the title and content text based on the arguments. 3. Applies custom styling using styleDialog and waits for the user to close the alert.
- void styleDialog(Dialog<> dialog)	A helper method that applies custom CSS to dialog boxes: 1. Retrieves the DialogPane from the dialog. 2. Adds the global stylesheet and the specific "game-dialog" CSS class to the pane. 3. Catches exceptions to prevent crashes if the CSS file is missing.
- void updateTurnDisplay(Button rollButton)	Updates the UI to reflect the current player's turn: 1. Retrieves the current player from the controller. 2. Updates the playerTurnLabel with the

	player's name. 3. Resets the roll button text and re-enables it. 4. Changes the label's background color (LimeGreen for Player 1, Red for Player 2).
- String toHexString(Color color)	A utility method that converts a JavaFX Color object into a standard Hexadecimal string format (e.g., "#RRGGBB") for use in CSS styling.
+ void updatePlayerTokens()	Refreshes the visual position of all player tokens on the board: 1. Removes all token images from their previous locations in the grid. 2. Iterates through each player to determine their current logical position. 3. Calculates the corresponding row and column indices on the grid. 4. Adds the token image to the target grid cell, aligning Player 1 to the bottom-left and Player 2 to the bottom-right to prevent overlap.
- StackPane getCellFromGrid(int row, int intcol)	A helper method that retrieves the specific StackPane (grid cell) located at the given row and column indices by iterating through the children of the boardGrid.
- VBox createSquareBox(Square square)	Helper method. Constructs the visual container for a single board square by performing the following actions: 1. Creates a styled label with the square's name. 2. Generates an icon for the square using createSquareIcon. 3. Arranges the icon and label within a VBox. 4. Calls updateSquareBoxStyle to apply the initial background colors and borders based on the square type.
- void updateSquareBoxStyle(VBox box, Square square)	Helper method. Applies CSS styling to the square's container based on its type and state: 1. Assigns specific background colors to special squares (Go, Jail, Card, etc.). 2. For properties, checks ownership status to apply player-specific colors (Green for P1, Red for P2). 3. Adjusts the border color and thickness (e.g., Gold thick border) if the property has been upgraded to a Landmark.

- void updateSquareBox(Square square)	Refreshes the visual representation of a specific square during gameplay (e.g., after a purchase or upgrade): 1. Locates the specific cell in the grid layout. 2. Clears the existing content of the square's container. 3. Regenerates the icon and label to reflect the new state (e.g., changing a generic house icon to a Landmark icon). 4. Reapplies the styling via updateSquareBoxStyle to update colors and borders.
- GridPane createBoardGrid()	Constructs the visual grid layout for the game board: 1. Initializes a 6x6 GridPane. 2. Iterates through every cell in the grid and calculates the corresponding logical square index using getIndexFromGrid. 3. If the index corresponds to a valid board square, calls createSquareBox to generate the visual content and adds it to the grid. 4. Leaves the inner cells of the grid empty to create the loop layout.
- int getIndexFromGrid(int row, int col, int dim)	Helper method. Maps the 2D grid coordinates (row and column) to the linear logical index (0-19) of the board squares. Returns -1 if the coordinates correspond to the empty center of the board.
- int getRowForPosition(int squareIndex)	Helper method. Determines the GridPane row index for a specific logical board position (0-19), enabling the placement of tokens or updates on the correct row.
- int getColForPosition(int squareIndex)	Helper method. Determines the GridPane column index for a specific logical board position (0-19), enabling the placement of tokens or updates on the correct column.
- ImageView createCharacterImageView(int index)	Helper method. Creates an ImageView for a character by: 1. Retrieving the filename based on the provided index. 2. Loading the image resource stream. 3. configuring the image view properties (width, preservation of ratio). 4. Handling exceptions if the image file is not found.

- HBox createPlayerPanel(Player player, int charIndex, boolean isPlayer1)	Helper method. Constructs the visual information panel for a player by performing the following actions: 1. Creates an ImageView for the player's character. 2. Creates Labels for the player's name and current money balance. 3. Assigns the money label to the class field (player1MoneyLabel or player2MoneyLabel) to allow for future updates. 4. Styles the panel with a specific background color (LightGreen for Player 1, LightCoral for Player 2).
+ void updatePlayerMoney(Player player)	Updates the displayed money balance for a specific player: 1. Identifies the correct UI label corresponding to the player. 2. Compares the currently displayed value with the player's actual balance. 3. If a difference is detected: plays a sound effect ("lottery.mp3") and updates the label text to reflect the new balance.
- ImageView createSquareIcon(Square square)	Helper method. Generates the appropriate icon for a square based on its type and state: 1. For Properties: Checks if it is a Landmark (returns "landmark.png") or if it is merely owned (returns "house.png"). 2. For Special Squares: Assigns specific icons for Lottery, Tax, Jail, Airplane, and Card squares. 3. Loads the image file and returns it as an ImageView.
- void playDiceSound()	Helper method. Loads and plays the dice rolling sound effect ("dice.mp3") using the AudioClip class. Includes error handling to log a message if the sound file cannot be found.
- void playSound(String fileName)	A helper method that plays a specific sound effect: 1. Constructs the resource URL by appending the filename to the /soundfx/ path. 2. Verifies if the resource exists; if found, plays the audio clip at full volume (1.0). 3. Logs error messages to the standard

	error stream if the file is not found or an exception occurs.
- void playSquareSound(Square square)	Determines and plays the appropriate sound effect based on the type of square the player landed on: 1. Checks the instance type of the square to select the corresponding sound file (e.g., "card.mp3", "jail.mp3", "lottery.mp3", or "plane.mp3"). 2. If a valid sound file is identified, attempts to load it from the /soundfx/ directory. 3. Plays the sound and prints debug logs to the console indicating success or failure.
- int getMoneyFromLabel(Label label)	A utility method that extracts the numeric money value from a UI label: 1. Retrieves the text content from the provided label (e.g., "Money: $2000"). 2. Removes all characters except digits and the minus sign using a regular expression. 3. Parses the resulting string into an integer and returns it (returns 0 if parsing fails).
- handleTestMove(int steps)	1. Checks the state of the "Roll Dice!" button. 2. Forces the Dice Result (gameController.getDice().setForcedResult(steps)) to the specified number of steps. 3. Disables the roll button and calls executeRollAndMove to execute the movement animation.
- handleRentableAction(Player player, Square square, Button rollButton)	1. Determine Ownership: Checks if the square (Property or IslandSquare) is owned. 2. Execute Case Logic: * Owned by Others: Calls handleRentPayment(). * Owned by Self: If it's a Property and not a Landmark, offers showUpgradePropertyDialog(). If it's an IslandSquare or already a Landmark, calls endTurn(). * Unowned: Offers the player to buy the property via showBuyPropertyDialog().
- handleRentPayment(Player player, Square square, Rentable rentable, Player owner, Button rollButton)	1. Rent Payment: Displays a mandatory rent Alert. Deducts the rent from the player and transfers it to the owner. 2. Bankruptcy: If the player cannot pay the full rent, they pay all remaining money, and the financial display is refreshed. 3. Buyout Opportunity (Property only): * If the property is not a Landmark, the player is offered to buy it out at 1.5 times the original price. * If accepted, money is transferred,

	ownership is changed, and the Landmark status is reset to false. 4. Turn End: Calls endTurn() to conclude the current player's action and advance the game

3.3 Class GameMenu

Fields and Constructors

- final Stage primaryStage	The main window (stage) of the application where the menu scene will be displayed.
- static final String COVER_IMAGE_PATH	A constant string representing the file path to the game's cover image ("/Image/bg.jpg").
+ GameMenu(Stage primaryStage)	Constructor method. Initializes the GameMenu instance with the provided primaryStage.

Methods

+ Scene createMenuScene()	Creates and returns the main menu scene by performing the following actions: 1. Starts playing the main menu background music ("main.mp3"). 2. Loads and configures the cover image, adjusting its size to fit the screen layout. 3. Creates UI components including the title label ("เกมยาจก"), "Start Game" button, and "Exit" button with specific CSS styles. 4. Configures the "Start Game" button to initialize the GameController, create the CharacterSelectScreen, and switch scenes. 5. Configures the "Exit" button to close the application. 6. Arranges all elements vertically in a VBox layout, applies the global CSS stylesheet, and returns the constructed Scene.

3.4 Class Main

Fields

+ static final int WIDTH	A constant defining the width of the application window (1000 pixels).
+ static final int HEIGHT	A constant defining the height of the application window (800 pixels).

Methods

+ void start(Stage primaryStage)	The main entry point for the JavaFX application. It initializes the UI by performing the following actions: 1. Creates a new instance of GameMenu. 2. Sets the window title to "เกมยาจก". 3. Sets the scene to the menu scene created by GameMenu.

	4. Displays the primary stage (window).
+ static void main(String[] args)	The standard main method for the Java application. It calls `launch(args)` to start the JavaFX application lifecycle.

3.5 Class Soundmanager

Fields and Constructors

- static MediaPlayer currentBGM	A static MediaPlayer instance used to control and track the currently playing background music to ensure only one track plays at a time.

Methods

+ static void playBGM(String fileName)	Plays a new background music track by performing the following actions: 1. Stops and disposes of any currently playing BGM to release resources. 2. Loads the specified audio file from the /soundfx/ resource path. 3. Creates a new MediaPlayer instance. 4. Configures the player to loop indefinitely (INDEFINITE). 5. Sets the volume to 0.5 and starts playback.
+ static void stopBGM()	Stops the currently playing background music if an active instance exists.

Link JavaDoc
https://github.com/2110215-ProgMeth/project-cp-2025-1-meth-addiction/tree/main/JavaDoc